

CHAPTER 1 .INTRODUCTION

CleanRidex is an online Car Wash Booking System designed to simplify and modernize the process of booking car wash services. In today's busy lifestyle, vehicle owners often find it difficult to allocate time for routine car cleaning through traditional methods that involve manual booking, phone calls, or long waiting queues. CleanRidex addresses this problem by providing a digital platform where customers can conveniently schedule car wash services anytime and from anywhere.

The system allows users to register, log in, view available car wash services, choose preferred service packages, select suitable dates and time slots, and make bookings with ease. Service providers can manage service listings, monitor bookings, update availability, and track customer requests efficiently through the system. This helps improve operational efficiency and ensures better coordination between customers and service providers.

CleanRidex aims to reduce waiting time, enhance transparency in pricing and service availability, and improve overall customer satisfaction. By automating the booking and management process, the system promotes digital adoption in the car wash industry while delivering a smooth, reliable, and user-friendly experience for both customers and service providers.

1.1 ABOUT THE PROJECT

CleanRidex is a web-based Car Wash Booking System developed to provide a smart and efficient solution for booking car wash services online. The project focuses on eliminating the drawbacks of traditional car wash booking methods, such as long waiting times, manual scheduling, and lack of service transparency. By digitizing the entire process, CleanRidex offers a seamless experience for both customers and service providers.

The system enables customers to easily register and log in, browse different car wash services and packages, check availability, select preferred time slots, and book services without visiting the service centre physically. Customers can also view booking status and manage their appointments, ensuring better convenience and time management.

From the service provider's perspective, CleanRidex helps manage service offerings, schedules, customer bookings, and availability efficiently. The system reduces manual workload, minimizes booking conflicts, and improves service planning. Automated records also help in maintaining accurate customer and service data.

Overall, CleanRidex aims to enhance customer satisfaction, optimize service operations, and promote digital transformation in the car wash industry. The project demonstrates the practical application of software engineering concepts and serves as a scalable solution that can be extended with features like online payment, service tracking, notifications, and feedback systems in the future.

1.2 OBJECTIVE AND SCOPE OF THE PROJECT

OBJECTIVE OF THE PROJECT :

The main objectives of the CleanRidex project are:

- To provide an easy-to-use online platform for booking car wash services.
- To reduce customer waiting time by enabling advance scheduling of services.
- To eliminate manual booking processes and improve accuracy in service management.
- To ensure transparency in service availability, pricing, and booking status.
- To help service providers manage bookings, schedules, and customer data efficiently.
- To improve overall customer satisfaction through a fast and reliable digital system.
- To promote digital adoption in small and medium-scale car wash service businesses.

SCOPE OF THE PROJECT :

The scope of the CleanRidex project includes the following functionalities:

- User registration and secure login for customers and service providers.
- Display of available car wash services and service packages with pricing details.
- Selection of preferred date and time slots for booking car wash services.
- Online booking and appointment management system.
- Booking status tracking for customers.
- Admin/service provider dashboard to manage services, schedules, and bookings.
- Storage and management of customer and booking details in a centralized database.

1.3 DEFINITION OF PROBLEM

The traditional car wash booking process is largely manual and inefficient, requiring customers to visit service centres directly or make phone calls to schedule services. This approach often results in long waiting times, lack of proper scheduling, overbooking or underutilization of service slots, and poor communication between customers and service providers. Customers also face difficulty in knowing service availability, pricing details, and booking status in advance.

From the service provider's perspective, managing appointments, customer records, and service schedules manually is time-consuming and error-prone. The absence of a centralized system makes it difficult to track bookings accurately, plan resources efficiently, and maintain service quality.

Therefore, there is a need for a reliable and user-friendly digital solution that automates the car wash booking process, provides real-time service availability, ensures transparency, and improves overall efficiency. CleanRidex is proposed to address these issues by offering an online car wash booking platform that streamlines service scheduling, enhances customer convenience, and supports efficient service management.

CHAPTER 2 : SYSTEM ANALYSIS

System analysis is the process of studying the existing problems and identifying the requirements of the proposed system. For the CleanRidex project, system analysis focuses on understanding the limitations of the current manual car wash booking process and defining how the new system will overcome them.

2.1 EXISTING SYSTEM

In the existing system, car wash services are booked manually through physical visits or phone calls. This system has several drawbacks:

- Customers must wait in queues or repeatedly call service centres to check availability.
- There is no proper scheduling mechanism, leading to overbooking or idle service slots.
- Service details, pricing, and availability are not clearly communicated to customers.
- Manual record-keeping increases the chances of errors and data loss.
- Service providers find it difficult to manage customer details, bookings, and daily schedules efficiently.
- Overall, the existing system is time-consuming, inefficient, and lacks transparency.

2.1.1 DISADVANTAGES

The CleanRidex system has certain limitations and disadvantages:

- Internet Dependency

The system requires a stable internet connection. Users in areas with poor connectivity may face difficulty accessing the service.

- Initial Setup Cost

Developing and deploying the system involves initial costs for software development, hosting, and maintenance.

- Technical Issues

System failures, server downtime, or software bugs can temporarily disrupt booking and service management.

- User Adaptation

Some customers or service providers may find it difficult to adapt to a digital system, especially those unfamiliar with technology.

- Security Risks

As the system stores user and booking data, it may be vulnerable to data breaches if proper security measures are not implemented.

- Limited Offline Access

The system cannot be used offline, which may be inconvenient in emergency or network failure situations.

- Maintenance Requirement

Regular updates and maintenance are required to ensure smooth functioning and security of the system.

2.2 PROPOSED SYSTEM

The proposed system, CleanRidex, is a web-based car wash booking system designed to automate the complete booking and service management process. It provides a centralized and user-friendly platform that connects customers with car wash service providers, ensuring efficiency, transparency, and convenience.

The system allows customers to register and log in securely, enabling personalized access to services. Users can view available car wash services and packages along with pricing details, select suitable dates and time slots, and book appointments online without visiting the service center. Customers can also track the status of their bookings, which helps reduce uncertainty and waiting time.

For service providers and administrators, CleanRidex offers a dedicated dashboard to manage service listings, schedules, and customer bookings effectively. This dashboard helps avoid overbooking, improves resource utilization, and simplifies daily operations. All customer and booking information is stored securely in a centralized database, ensuring data accuracy, consistency, and reliability.

Overall, the proposed system replaces manual and time-consuming booking methods with an efficient digital solution, improving customer satisfaction and enabling service providers to manage operations smoothly and systematically.

2.2.1 ADVANTAGES

Advantages of the Proposed System

- Reduces waiting time and manual effort.
- Improves accuracy and efficiency in booking management.
- Provides transparency in pricing and service availability.
- Enhances customer convenience and satisfaction.
- Helps service providers manage operations effectively.
- Scalable and can be enhanced with future features.

2.2.2 MODULES

The proposed system, CleanRidex, is divided into well-defined modules to ensure efficient operation, easy maintenance, and scalability. Each module is responsible for a specific set of functionalities within the system.

1. Customer Module

This module allows users to interact with the system and book car wash services.

Functions:

- User registration and secure login
- View available car wash services and packages
- Select preferred date and time slots
- Book car wash services
- View and track booking status

2. Service Provider Module

This module enables service providers to manage services and bookings efficiently.

Functions:

- Add, update, and delete car wash services
- Set service availability and time slots
- View customer booking requests
- Update booking status

3. Admin Module

This module provides complete control over the system.

Functions:

- Manage users and service providers
- Monitor all bookings
- Manage system settings and services
- Generate reports

4. Booking Management Module

This module handles the core booking functionality.

Functions:

- Process service booking requests
- Manage appointment scheduling
- Prevent time-slot conflicts
- Maintain booking records

5. Payment Module

This module supports secure online payments.

Functions:

- Online payment processing
- Payment confirmation
- Transaction history

6. Notification Module

This module keeps users informed about booking activities.

Functions:

- Booking confirmation notifications
- Booking status updates
- Reminder alerts

7. Database Management Module

This module ensures secure data storage and retrieval

Functions:

- Store user, service, booking, and payment data
- Maintain data security and integrity

2.3 FEASIBILITY STUDY

A feasibility study is conducted to determine whether the proposed system is practical, achievable, and beneficial. For the CleanRidex project, the feasibility study evaluates the system from technical, economic, operational, and legal perspectives.

1. Technical Feasibility

The CleanRidex system is technically feasible as it is developed using the MERN stack (MongoDB, Express.js, React.js, Node.js), which is widely used for modern web applications. These technologies are open-source, well-supported, and compatible with standard hardware and software environments. The system can run on commonly available web servers and browsers without requiring specialized equipment. Hence, the technical resources required for development and deployment are readily available.

2. Economic Feasibility

The project is economically feasible because it minimizes development and operational costs. The MERN stack uses free and open-source tools, reducing licensing expenses. Automation of booking and management processes reduces manual effort and operational overhead for service providers. In the long run, the system helps increase efficiency and customer satisfaction, leading to better revenue opportunities.

3. Operational Feasibility

CleanRidex is operationally feasible as it is designed to be user-friendly and easy to use. Customers can book services with minimal steps, and service providers can manage bookings through a simple dashboard. The system does not require extensive training, making it suitable for users with basic technical knowledge. Therefore, it can be easily adopted in real-world environments.

4. Legal Feasibility

The system complies with standard data protection and privacy guidelines. User information is stored securely, and access control mechanisms are implemented to prevent unauthorized access. As the system does not violate any legal or regulatory requirements, it is legally feasible.

5. Schedule Feasibility

The project can be completed within the stipulated time using proper planning and modular development. The use of modern frameworks and reusable components helps speed up development and testing.

CHAPTER 3: METHADODOLOGY

3.1 AGILE SOFTWARE DEVELOPMENT

Agile Software Development is an iterative and incremental methodology that emphasizes flexibility, continuous user involvement, and frequent delivery of functional software. Unlike traditional sequential development models, Agile allows software to be developed in small, manageable iterations, enabling the development team to respond effectively to changing requirements throughout the project lifecycle.

For the CleanRidex – Car Wash Booking System, the Agile approach is particularly suitable because the application is designed to serve customers, service providers, and administrators whose requirements may evolve based on real-world usage. As users interact with features such as service selection, booking schedules, and booking status management, new usability requirements and functional improvements may arise. Agile development enables the team to incorporate these changes efficiently without affecting the overall system architecture.

Agile supports the frequent delivery of working software, which allowed CleanRidex to be developed and evaluated in multiple stages. The project initially focused on a minimum viable product (MVP) that included core functionalities such as user registration, login, viewing car wash services, and booking appointments. Subsequent iterations introduced enhancements such as booking status tracking, service provider management, and administrative controls. This incremental approach ensured early validation of essential features while allowing continuous refinement based on user feedback.

Another important aspect of Agile in the CleanRidex project is its emphasis on customer collaboration. Feedback from users and service providers was considered to improve booking workflows, time-slot selection, and service management. User interactions helped refine the interface and simplify navigation, ensuring ease of use for users with varying levels of technical knowledge.

Agile also promotes adaptability by allowing development priorities to shift according to actual usage patterns, system performance, and stakeholder feedback. Features were

refined and prioritized based on practical requirements, ensuring that the system aligned closely with real-world service booking needs rather than rigid initial assumptions.

Finally, Agile contributes to risk reduction by encouraging early testing and validation of individual components. By developing and testing features incrementally, potential design and technical issues were identified and resolved early. The Agile methodology, supported by short development iterations, continuous backlog refinement, clearly defined acceptance criteria, and a structured Definition of Done, provided a flexible yet disciplined framework for the successful development of the CleanRidex application.

3.2 SCRUM FRAMEWORK

The Scrum framework was adopted in this project to support Agile development and ensure effective collaboration. Scrum defines specific roles and processes that help streamline development and improve productivity.

Scrum Roles

1.Product Owner (PO):

The Product Owner represents stakeholders and defines business requirements for the CleanRidex system. The Product Owner is responsible for creating, prioritizing, and maintaining the product backlog to ensure that high-value features such as booking management and service scheduling are developed first.

2.Scrum Master:

The Scrum Master facilitates the Scrum process by ensuring Agile principles are followed. The Scrum Master removes obstacles that may hinder development and supports the team in achieving sprint goals efficiently.

3.Development Team:

The Development Team is a cross-functional and self-organizing group responsible for analysis, design, development, testing, and integration of the CleanRidex system. The team collaborates to deliver a functional increment at the end of each sprint.

The CleanRidex project was developed using the Scrum framework, with each sprint focusing on specific system modules such as user authentication, service management,

booking processing, and administrative controls. This iterative approach ensured continuous progress, early validation, and flexibility to incorporate improvements.

Key Sprints Included

1. User Authentication & Profile Management Sprint

This sprint focused on implementing secure user registration and login for customers, service providers, and administrators. It included role-based access control and authentication mechanisms to ensure secure system usage.

2. Car Wash Service Management Sprint

This sprint addressed service-related functionalities. Service providers could add, update, delete, and manage car wash services and pricing. The sprint ensured accurate service listings for customer browsing.

3. Booking & Appointment Management Sprint

This sprint enabled customers to book car wash services by selecting preferred date and time slots. It also allowed service providers and admins to view, confirm, or update booking statuses.

4. Booking Status & Tracking Sprint

This sprint focused on booking status management, allowing users to track their booking progress. Clear status indicators such as pending, confirmed, and completed were implemented to improve transparency.

5. UI Refinement & Data Integrity Sprint

This sprint concentrated on improving system usability through UI enhancements, input validation, and performance optimization. Feedback-driven refinements improved navigation, form simplicity, and overall user experience.

Scrum Framework Implementation

The use of the Scrum framework enabled CleanRidex to maintain an adaptive and collaborative development workflow. Each sprint was time-boxed and aimed to deliver

a complete, functional, and testable set of features. Sprint planning sessions prioritized backlog items based on user needs and business value.

Sprint review meetings were conducted at the end of each sprint to demonstrate completed features and gather feedback. Feedback collected during these reviews was incorporated into subsequent sprints, enabling continuous improvement. Sprint retrospectives helped identify challenges, improve estimation accuracy, and refine development practices.

Several key refinements emerged from retrospectives:

1. Booking Workflow Improvements

Enhancements were made to the booking process, including clearer time-slot selection and confirmation messages, improving user clarity and reducing booking errors.

2. Service & Status Management Enhancements

Booking status indicators and service management views were refined to improve coordination between customers and service providers.

3. UI & Navigation Refinements

UI workflows were simplified through cleaner layouts, clearer buttons, and improved form validation, making the system accessible even to users with limited technical expertise.

3.3 USER STORY

User stories played a crucial role in guiding the development of the CleanRidex – Car Wash Booking System by ensuring that each feature directly addressed the real needs of its stakeholders. Every user story clearly defines who requires a particular feature, what functionality is needed, and why it is important. These user stories helped align system development with real-world challenges faced by customers, service providers, and administrators in managing car wash services and bookings efficiently.

1.Admin User Stories

- As an admin, I want to manage user and service provider accounts so that only authorized users can access the system.
- As an admin, I want to monitor booking activities and system usage so that irregular or suspicious activities can be identified early.
- As an admin, I want to activate, deactivate, or block user and service provider accounts so that platform security and data integrity are maintained.
- As an admin, I want to view booking reports and system feedback so that improvements can be planned to enhance system performance and usability.

2.Service Provider User Stories

- As a service provider, I want to register and manage my service profile so that customers can view accurate service information.
- As a service provider, I want to add, update, and remove car wash services so that service listings remain up to date.
- As a service provider, I want to view and manage customer booking requests so that appointments can be handled efficiently.
- As a service provider, I want to confirm, reschedule, or cancel bookings so that service schedules are managed effectively.
- As a service provider, I want to view booking history and daily schedules so that service operations can be planned properly.

3. Customer User Stories

- As a customer, I want to register and log in securely so that my account information is protected.
- As a customer, I want to browse available car wash services and packages so that I can select a suitable service.
- As a customer, I want to book car wash services by selecting preferred date and time slots so that I can avoid waiting in queues.
- As a customer, I want to track the status of my bookings so that I am informed about confirmations and service progress.
- As a customer, I want to receive notifications regarding booking confirmations, updates, or cancellations so that I stay informed.

3.4 SPRINT

Sprint planning played a vital role in the development of the CleanRidex – Car Wash Booking System by dividing the project into clear, time-boxed development phases. Each sprint focused on implementing specific functional requirements that addressed real-world needs of customers, service providers, and administrators. This approach ensured continuous progress, early validation, and the effective delivery of a reliable car wash booking platform.

Sprint Planning

During sprint planning, the development team collaboratively identified and prioritized features based on their importance to system usability and business value. High-priority functionalities such as user authentication, service management, booking processing, booking status tracking, and administrative controls were selected for implementation.

User stories were decomposed into smaller development tasks and assigned to activities such as backend development, frontend design, database implementation, and testing. Each task was estimated and added to the sprint backlog, with objectives clearly aligned to the roles of Admin, Service Provider, and Customer.

Sprint Execution and Review

Sprint execution involved the step-by-step development of core features including secure user authentication, car wash service management, booking and appointment handling, and booking status tracking. Regular progress tracking helped monitor development status and resolve issues efficiently.

At the end of each sprint, a sprint review session was conducted where completed features were demonstrated and evaluated. Feedback received during these reviews was used to refine existing functionalities and plan improvements for subsequent sprints.

Incremental Delivery

Each sprint resulted in a functional and testable increment of the CleanRidex system, enabling early feedback and validation. For example, after implementing the booking module, refinements were made to improve time-slot selection clarity

and booking confirmation messages. This incremental delivery approach reduced technical risks, ensured early delivery of essential features, and supported continuous improvement based on real user feedback.

Code	As a/an	I want to	So that I can
US 1	Admin	Log into the system securely	Access and manage the platform effectively
US 2	Admin	Manage users and service providers	Ensure only authorized users access the system
US 3	Admin	Monitor and block or deactivate users	Maintain system integrity
US 4	Service Provider	Register and manage service profile	Provide accurate service information
US 5	Service Provider	Add, update, and delete car wash services	Keep service listings up to date
US 6	Service Provider	View and manage booking requests	Process appointments efficiently
US 7	Service Provider	Manage booking schedules	Avoid conflicts and delays
US 8	Service Provider	Update booking status	Keep customers informed
US 9	Customer	Register and log in securely	Protect my account information
US 10	Customer	Browse car wash services	Choose suitable services easily
US 11	Customer	Book car wash services online	Avoid waiting in queues
US 12	Customer	Track booking status	Stay informed about confirmations
US 13	Customer	Receive booking notifications	Manage appointments efficiently
US 14	Customer	Provide feedback	Help improve system efficiently

Table 3.1 Scrum Agile User Stories

Milestone	Completion Date
Problem Identification & Idea Validation	01/06/2025
User Story Collection & Requirements Analysis	12/06/2025
Product Backlog Creation	20/06/2025
High Level Sprint Planning	26/06/2025
UI Wireframes & Database Design	05/07/2025
Sprint 1 Completion	20/07/2025
Sprint 2 Completion	30/07/2025
Sprint 3 Completion	12/08/2025
Sprint 4 Completion	24/08/2025
Sprint 5 Completion	02/09/2025
Sprint 6 & Project Finalization	30/12/2025

Table 1 : MAJOR MILESTONES

Sprint Breakdown

Sprint 1 : User and Admin Management

- Password encryption and authentication
- Role-based access control (Admin/User)
- Profile management
- Account activation and deactivation
- Admin user management features
- Initial environment setup and database connection

Sprint 2: Service Management

- Service provider registration and profile management
- Add, update, and delete car wash services
- Service availability and pricing management
- Validation to prevent duplicate services
- Admin controls for service monitoring

Sprint 3: Booking & Appointment Management

- Booking creation with date and time slot selection

- Booking confirmation and cancellation
- Booking history and status tracking
- Validation to prevent slot conflicts

Sprint 4: Customer & Service Provider Interaction

- Service browsing and search
- Booking notifications and status updates
- Service provider schedule management
- Feedback collection from customers
- Admin monitoring and issue handling

Sprint 5: UI Enhancements & Optimization

- UI/UX improvements
- Input validation and error handling
- Performance optimization
- Improved booking summary views

Sprint 6: Testing, Optimization & Documentation

- End-to-end system testing
- Unit and integration testing
- Data integrity verification
- Bug fixing and final refinements
- Documentation preparation
- Version control finalization
- Project deployment and handover

CHAPTER 4 : SYSTEM REQUIREMENTS AND SPECIFICATIONS

3.1 HARDWARE CONFIGURATIONS

Processor: Intel Core i3 or higher

RAM: Minimum 4 GB (8 GB recommended)

Hard Disk: 20 GB free space

Device: Desktop/Laptop/Smartphone

Internet connection

3.2 SOFTWARE CONFIGURATIONS

Operating System: Windows / Linux / macOS

Frontend: React.js

Backend: Node.js, Express.js

Database: MongoDB

Web Browser: Google Chrome / Firefox / Edge

Code Editor: VS Code

Server: Localhost / Cloud server

3.3 TECHNOLOGY USED

Frontend: React.js

Used to create a responsive and interactive user interface for customers, service providers, and admin.

Backend: Node.js with Express.js Handles server-side logic, REST APIs, authentication, and communication between frontend and database.

Database: MongoDB

A NoSQL database used to store user details, service information, bookings, and schedules securely.

Programming Languages:

JavaScript (Frontend & Backend)

HTML, CSS (UI design)

Authentication:

JSON Web Tokens (JWT) for secure login and session management

3.4 PLATFORM USED

Operating System: Windows / Linux / macOS

Development Platform: MERN Stack

IDE / Editor: Visual Studio Code

Web Browser: Google Chrome / Mozilla Firefox

Server Platform: Localhost (Development), Cloud Server (Deployment)

Version Control: Git and GitHub

CHAPTER 5 : SYSTEM DESIGN

System design describes the overall structure of the CleanRidex system, explaining how different components interact to deliver the required functionality. The system follows a client–server architecture using the MERN stack.

4.1 DATABASE DESIGN

The database design of CleanRidex uses MongoDB, a NoSQL database, to store and manage data efficiently. Data is organized into collections, and relationships are maintained using unique IDs. This design ensures flexibility, scalability, and fast access to booking information.

TABLES

USER TABLE

Stores details about customers , service providers and admins.

Column Name	Data Type	Constraints	Description
User Id	Integer	PRIMARY KEY, AUTO_INCREMENT	Unique user identifier
name	VARCHAR (255)	NOT NULL	Users full name
email	Varchar (255)	UNIQUE, NOT NULL	User email (used as username)
password	VARCHAR (255)	NOT NULL	Hashed user password
phone	VARCHAR (20)	NULL	Contact number
role	VARCHAR (200)	NULL	Brief explanation of role
created _at	Date Time	AUTO_NOW_ADD	Account creation timestamp

SERVICE TABLE

Stores car wash services and packages offered by service providers

Column Name	Data Type	Constraints	Description
Service Id	Integer	PRIMARY KEY, AUTO_INCREMENT	Unique Identifier
Service Name	VARCHAR (100)	NOT NULL	Name of service
description	VARCHAR (255)	NULL	Details about the service
price	DECIMAL (10,2)	NOT NULL	Cost of service
duration	INTEGER	NOT NULL	Time required to complete the service
Provider Id	INTEGER	FOREIGN KEY,NOT NULL	References the service provider offering the service

BOOKING TABLE

Stores booking and appointment details.

Column Name	Data Type	Constraints	Description
Booking Id	INTEGER	PRIMARY KEY, AUTO_INCREMENT	Unique Identifier
User Id	INTEGER	FOREIGN KEY, NOT NULL	References the customer
Service Id	INTEGER	FOREIGN KEY, NOT NULL	References the selected carwash service

Booking Date	DATE	NOT NULL	Date
Time Slot	VARCHAR (50)	NOT NULL	Time slot chosen
Booking Status	VARCHAR (30)	DEFAULT 'Pending'	Status of booking

TIME SLOTS TABLE

Manages availability of service slots.

Column Name	Data Type	Constraints	Description
Slot Id	INTEGER	PRIMATY KEY, AUTO_INCREMENT	Unique Identifier
Service Id	INTEGER	FOREIGN KEY,NOT NULL	References the car wash service
date	DATE	NOT NULL	Date for which time slot is available
Start Time	TIME	NOT NULL	Start time of service slot
end Time	TIME	NOT NULL	End time
Availability Status	VARCHAR (30)	DEFAULT 'Available'	Slot status

PAYMENTS TABLE

Stores online payment details.

Column Name	Data Type	Constraints	Description
Payment Id	INTEGER	PRIMARY KEY, AUTO_INCREMENT	Unique Identifier

Booking Id	INTEGER	FOREIGN KEY, NOT NULL	References the related booking
amount	DECIMAL (10,2)	NOT NULL	Payment amount
Payment Method	VARCHAR (50)	NOT NULL	Mode of Payment
Payment Status	VARCHAR (30)	DEFAULT 'Pending'	Status of Payment
Transaction Date	DATETIME	DEFAULT CURRENT_TIMESTAMP	Date and Time of transaction

5.2 DATAFLOW DIAGRAM

5.2.1 DFD LEVEL 0

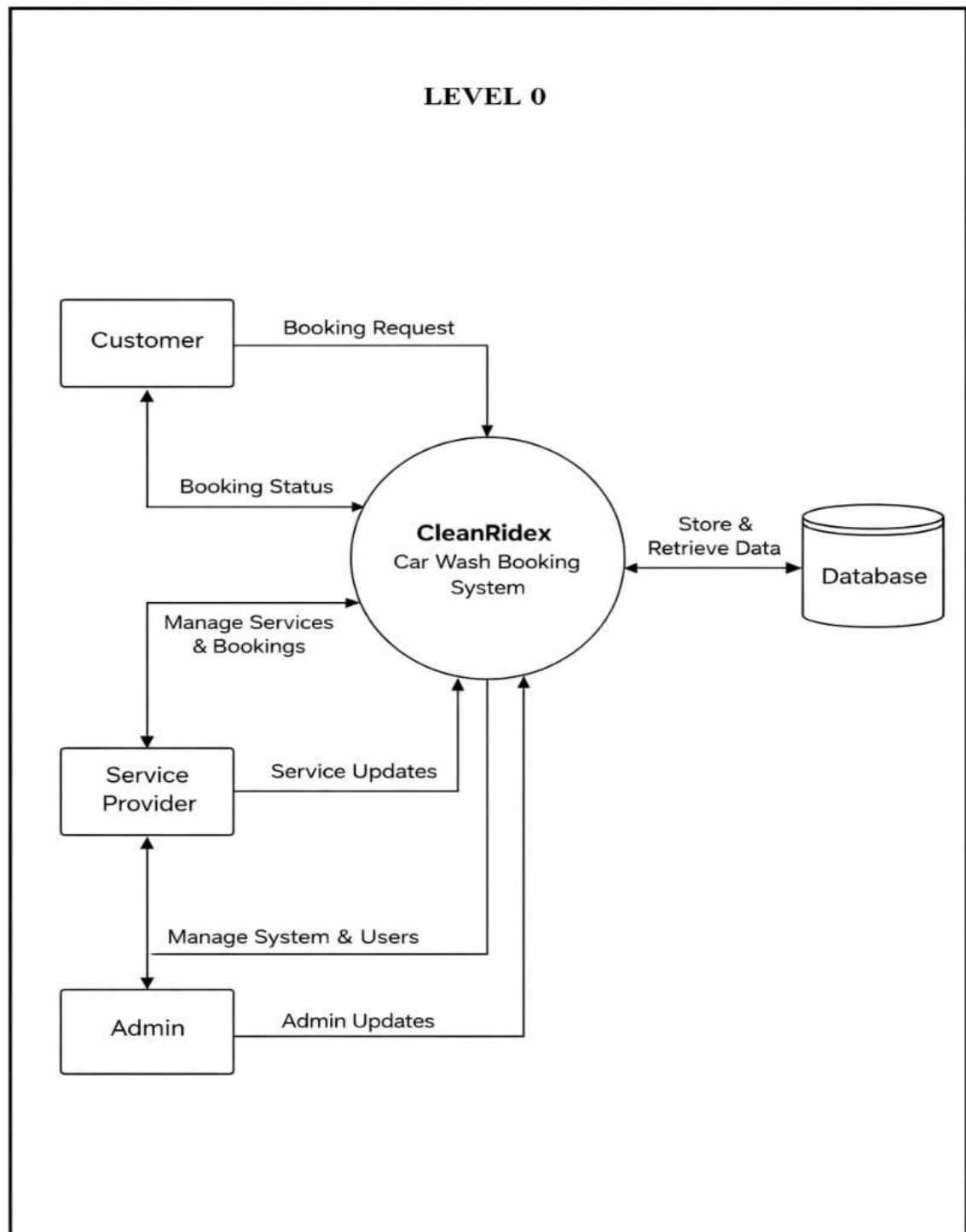


Figure 1: DFD LEVEL 0 – CLEANRIDEX – CAR WASH BOOKING SYSTEM

5.2.2 DFD LEVEL 1 ADMIN

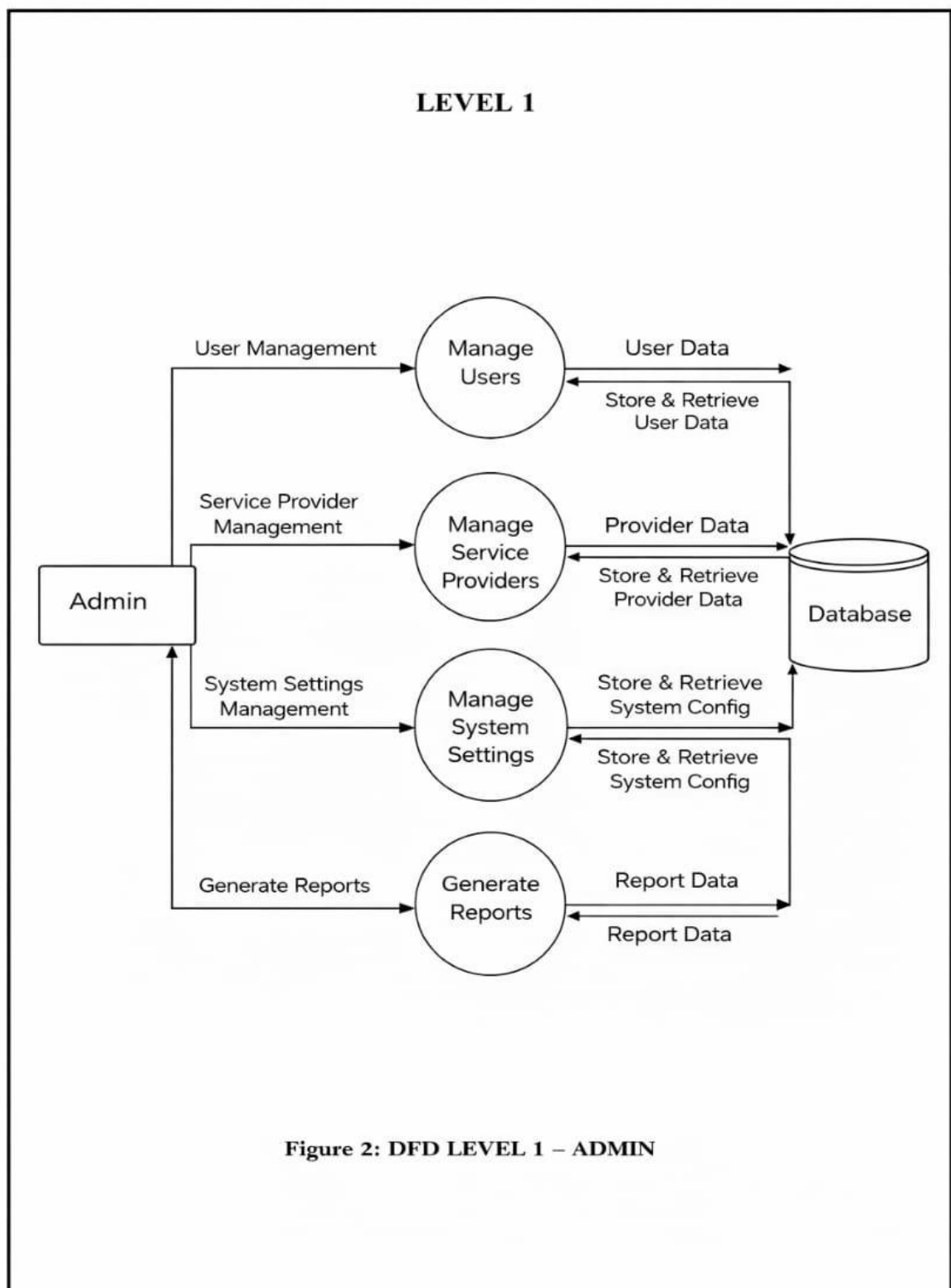


Figure 2: DFD LEVEL 1 – ADMIN

Figure 2: DFD LEVEL 1 – ADMIN

5.2.3 DFD LEVEL 2 USER

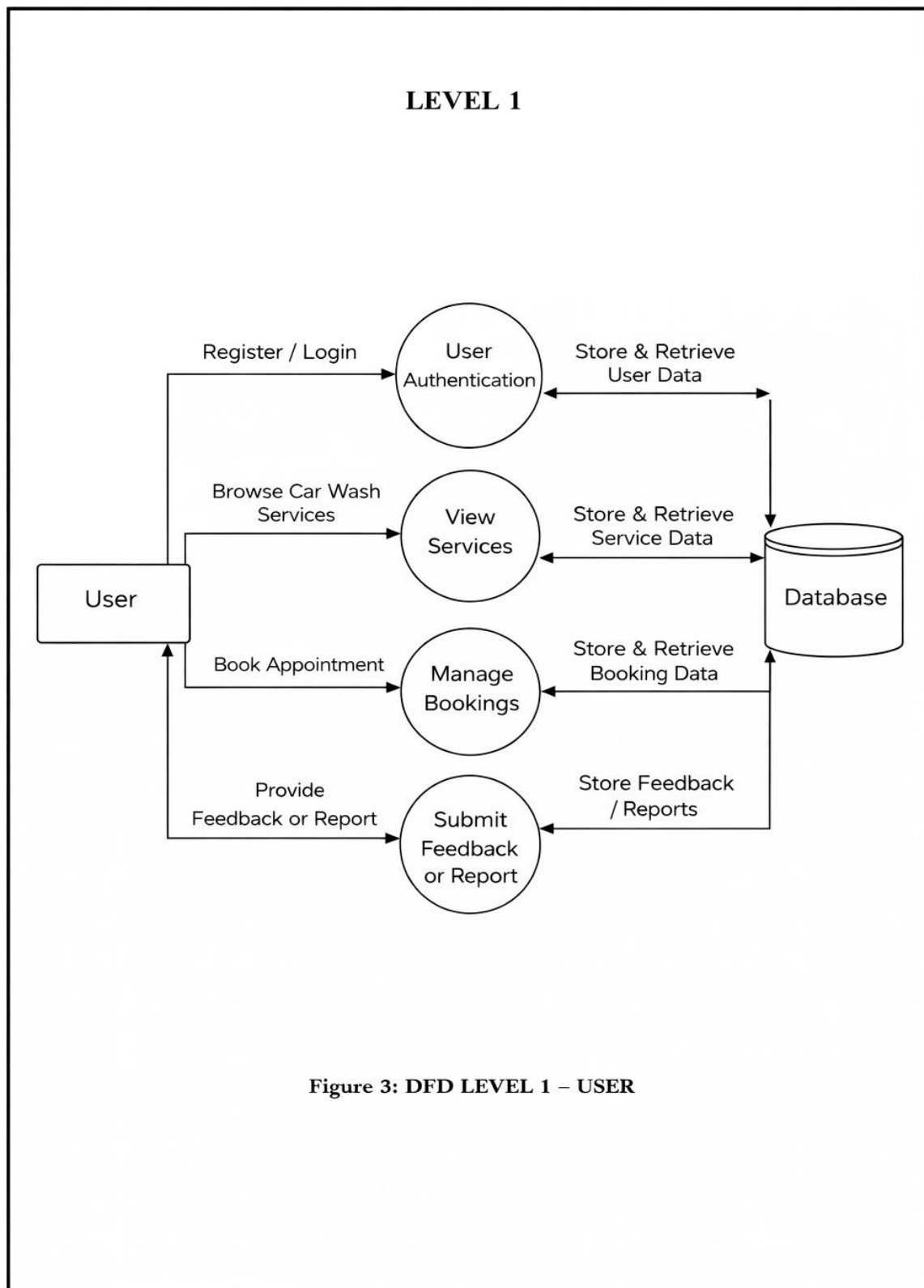
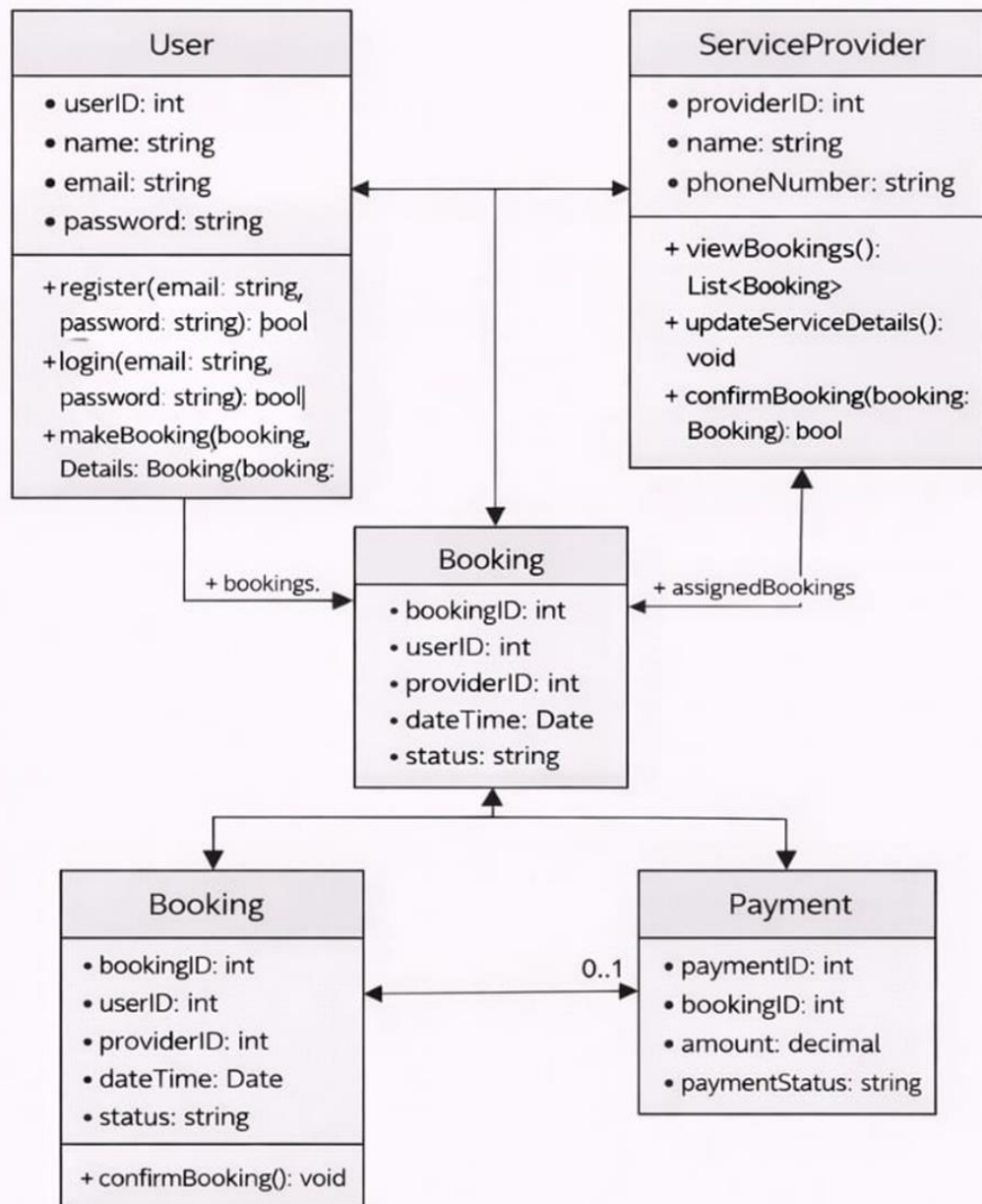


Figure 3: DFD LEVEL 1 – USER

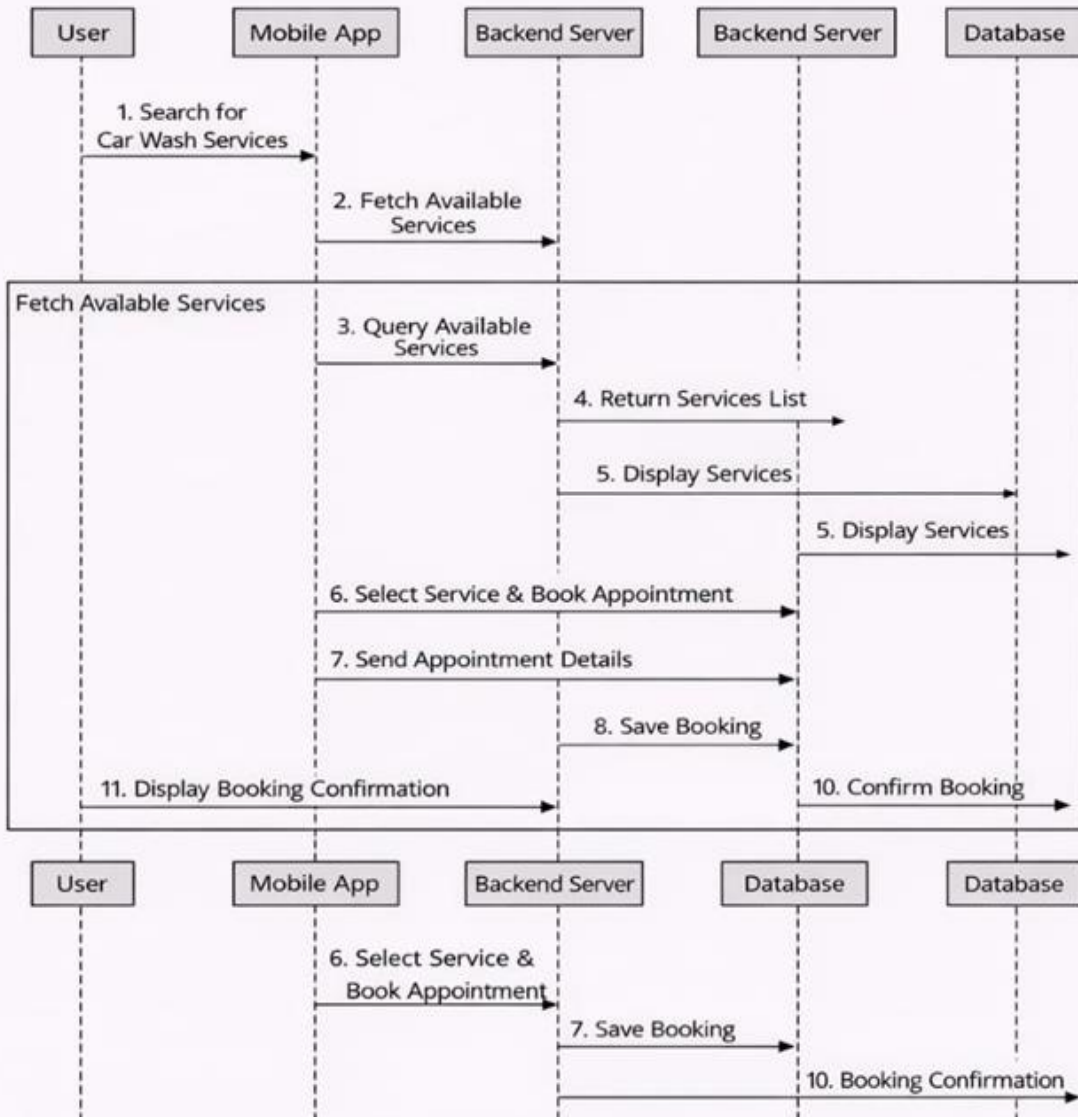
5.3 CLASS DIAGRAM

FIGURE: Class Diagram – CleanRidex – Car Wash Booking System



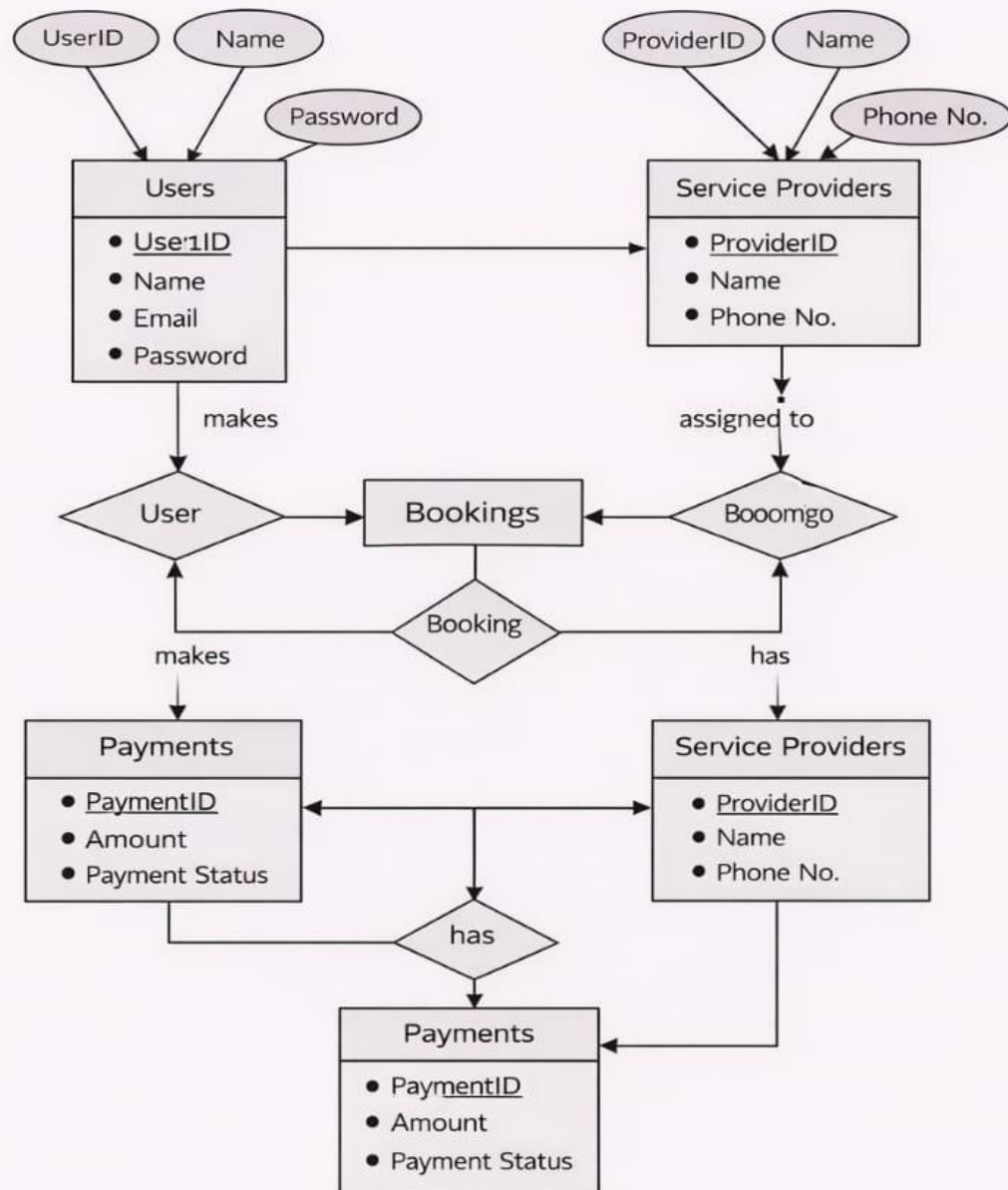
5.4 SEQUENCE DIAGRAM

FIGURE: Sequence Diagram – CleanRidex – Car Wash Booking System



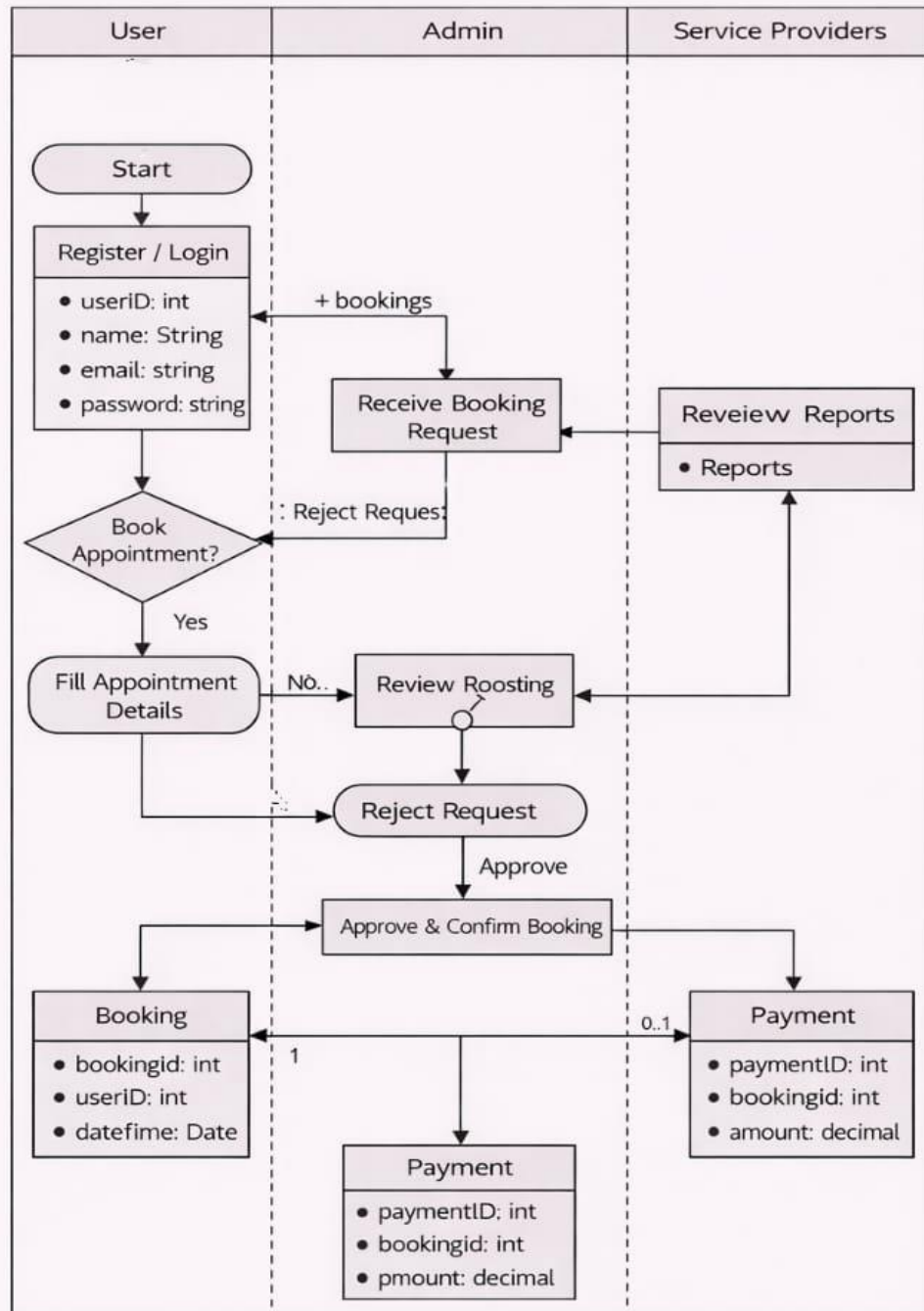
5.5 ER DIAGRAM

FIGURE: ER Diagram – CleanRidex – Car Wash Booking System



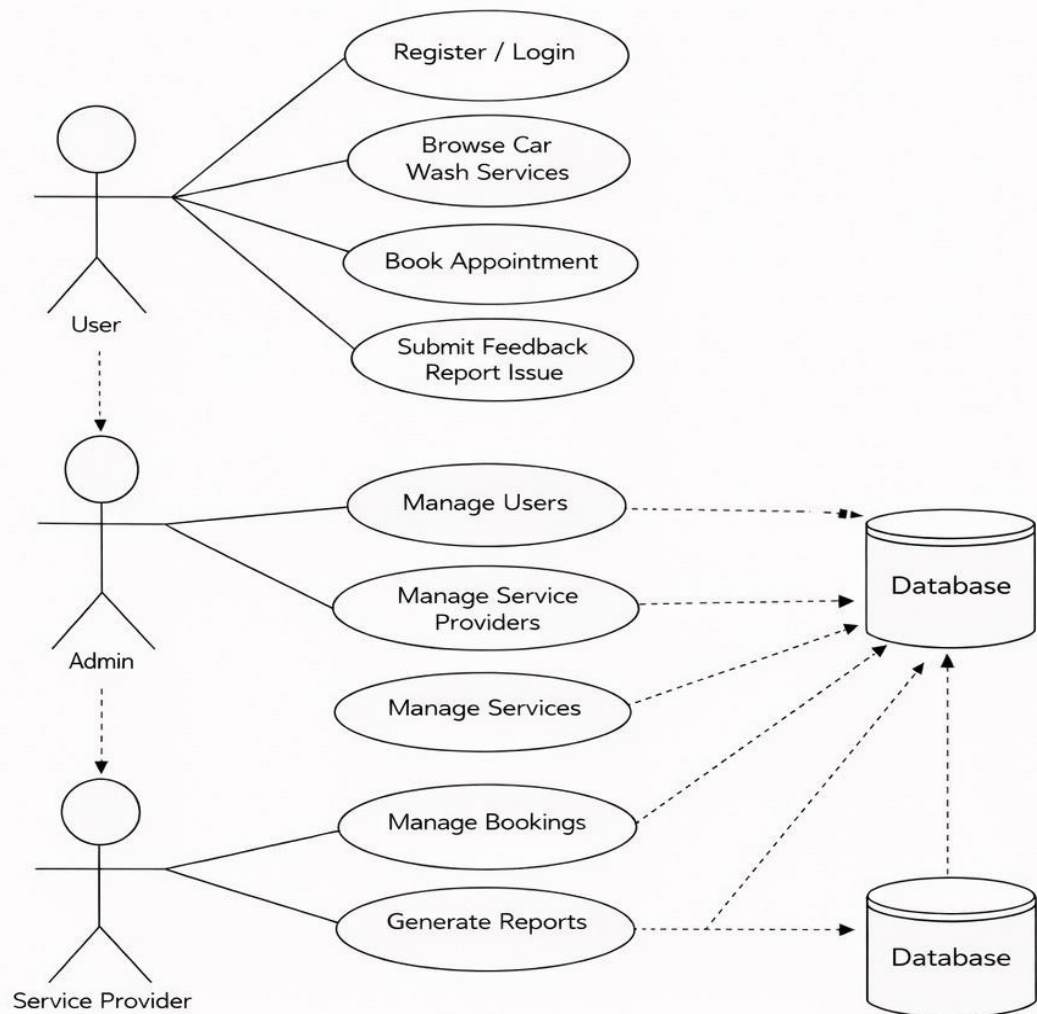
5.6 ACTIVITY DIAGRAM

FIGURE: Combined Activity Diagram – CleanRidex – Car Wash Booking System



5.7 USECASE DIAGRAM

USE CASE DIAGRAM – CleanRidex – Car Wash Booking System



CHAPTER 6:SYSTEM IMPLEMENTATION

,TESTING AND MAINTENANCE

System Implementation

System implementation refers to the phase where the designed system is converted into a working application. The CleanRidex system is implemented using the MERN stack, ensuring scalability, performance, and ease of maintenance.

1. Implementation Approach

The system follows a client–server architecture:

Frontend (Client Side):

- Developed using React.js, which provides a dynamic and responsive user interface. Components are created for user registration, login, service browsing, booking, and status tracking.

Backend (Server Side):

- Implemented using Node.js with Express.js. The backend handles business logic, RESTful API endpoints, authentication, and communication with the database.

Database Layer:

- MongoDB is used to store user details, service provider information, bookings, and payment records. Mongoose is used for schema definition and data validation.

2. Module-wise Implementation

-User Module

- Implemented user registration and login using JWT-based authentication.
- Users can browse services, book appointments, and view booking status.
- Service Provider Module
- Providers can log in and manage service details.
- View assigned bookings and confirm service completion.

- Admin Module
- Admin can manage users, service providers, and bookings.
- Generates reports and monitors system activities.

-Booking Module

- Handles booking creation, validation, and scheduling.
- Prevents time-slot conflicts and updates booking status.
- Payment Module (Option
- Designed for integration with online payment gateways.
- Stores transaction details securely.

3. Security Implementation

- Passwords are encrypted using hashing techniques.
- JWT is used for secure authentication and authorization.
- Role-based access control ensures restricted access to system features.

4. Testing and Deployment

- Unit testing is performed for individual modules.
- Integration testing ensures smooth communication between frontend and backend.
- The application can be deployed on a cloud platform with MongoDB Atlas for database hosting.

Testing

Testing is a critical phase in the software development life cycle that ensures the system works as intended and meets user requirements. For the CleanRidex project, testing was carried out at different levels to verify functionality, performance, and security.

1. Unit Testing

- Unit testing focuses on testing individual modules of the system separately.
- User registration and login validation
- Service listing and retrieval functions
- Booking creation and update functions

- Payment processing functions (if implemented)
- Each function was tested with valid and invalid inputs to ensure correct behavior.

2. Integration Testing

- Integration testing ensures that different modules work together properly.
- Interaction between frontend (React.js) and backend (Node.js/Express.js)
- Communication between backend and MongoDB database
- Booking flow from user request to confirmation by admin/service provider
- This testing helped identify issues related to data flow and API communication.

3. System Testing

- System testing evaluates the complete system as a whole.
- End-to-end booking process
- User authentication and authorization
- Booking status updates and confirmations
- Error handling and system response
- The system was tested against functional requirements to ensure all features work correctly.

4. Validation Testing

- Validation testing confirms that the system satisfies user needs.
- Ease of use and navigation
- Correct display of services and booking details
- Accurate booking confirmations
- This ensures the system is acceptable from the user's perspective.

5. Security Testing

- Security testing was conducted to protect user data.
- Password encryption testing
- Authentication and authorization checks
- Protection against unauthorized access

6. Performance Testing

- Performance testing checks system efficiency under load.
- Response time for booking requests
- Concurrent user access handling
- Database query performance

7. Testing Tools Used

- Postman for API testing
- Browser developer tools for frontend testing
- MongoDB Compass for database verification

Maintenance

Maintenance is the final phase of the software development life cycle and involves updating, modifying, and improving the system after deployment. The maintenance phase of CleanRidex ensures that the system remains reliable, secure, and efficient over time.

1. Corrective Maintenance

- Corrective maintenance focuses on fixing errors and bugs identified after the system is deployed.
- Resolving software bugs and runtime errors
- Fixing booking or payment-related issues
- Correcting data inconsistencies in the database

2. Adaptive Maintenance

- Adaptive maintenance involves modifying the system to adapt to changes in the environment.
- Updating the system to support new browsers or devices
- Modifying system features based on changing user requirements
- Ensuring compatibility with updated libraries and frameworks

3. Perfective Maintenance

- Perfective maintenance aims at enhancing system performance and usability.
- Improving user interface and user experience
- Optimizing database queries for faster performance
- Adding new features such as notifications and online payments

4. Preventive Maintenance

- Preventive maintenance helps reduce future system failures.
- Regular database backups
- Security updates and patch management
- Code refactoring to improve maintainability
- Monitoring server performance and logs

5. Maintenance Tools and Practices

- Version control using Git
- Continuous monitoring and logging
- Regular system updates and testing
- Documentation updates

CHAPTER 7 : SAMPLE CODE

Backend

- Index.js

```
const express = require('express');
const mongoose = require('mongoose');
const cors = require('cors');
require('dotenv').config();

const app = express();

// Middleware
app.use(cors());
app.use(express.json());
app.use(express.urlencoded({ extended: true }));
const paymentRoutes = require('./routes/payment.routes');
// Routes
app.use('/api/auth', require('./routes/auth'));
app.use('/api/services', require('./routes/services'));
app.use('/api/bookings', require('./routes/bookings'));
app.use('/api/reviews', require('./routes/reviews'));
app.use('/api/admin', require('./routes/admin'));
app.use('/api/payments', paymentRoutes);

// MongoDB Connection
const PORT = process.env.PORT || 5000;
const MONGODB_URI = process.env.MONGODB_URI ||
'mongodb://localhost:27017/carwash';

mongoose.connect(MONGODB_URI, {
  useNewUrlParser: true,
  useUnifiedTopology: true,
})
.then(() => {
  console.log('MongoDB connected successfully');
  app.listen(PORT, () => {
```



```

    console.log(`Server is running on port ${PORT}`);
  });
})
.catch((error) => {
  console.error('MongoDB connection error:', error);
  process.exit(1);
});

// Error handling middleware
app.use((err, req, res, next) => {
  console.error(err.stack);
  res.status(500).json({ message: 'Something went wrong!', error:
err.message });
});

```

- Review.js

```

const mongoose = require('mongoose');

const reviewSchema = new mongoose.Schema({
  customer: {
    type: mongoose.Schema.Types.ObjectId,
    ref: 'User',
    required: true
  },
  service: {
    type: mongoose.Schema.Types.ObjectId,
    ref: 'Service',
    required: true
  },
  booking: {
    type: mongoose.Schema.Types.ObjectId,
    ref: 'Booking',
    required: true
  }
});

```

```

    },
    rating: {
      type: Number,
      required: true,
      min: 1,
      max: 5
    },
    comment: {
      type: String,
      trim: true
    },
    createdAt: {
      type: Date,
      default: Date.now
    }
  }
});

module.exports = mongoose.model('Review', reviewSchema);

```

- User.js

```

const mongoose = require('mongoose');
const bcrypt = require('bcryptjs');

const userSchema = new mongoose.Schema({
  name: {
    type: String,
    required: [true, 'Please provide a name'],
    trim: true
  },
  email: {
    type: String,
    required: [true, 'Please provide an email'],
    unique: true,

```

```

        lowercase: true,
        trim: true
    },
    password: {
        type: String,
        required: [true, 'Please provide a password'],
        minlength: 6,
        select: false
    },
    phone: {
        type: String,
        required: [true, 'Please provide a phone number'],
        trim: true
    },
    role: {
        type: String,
        enum: ['customer', 'provider', 'admin'],
        default: 'customer'
    },
    address: {
        street: String,
        city: String,
        state: String,
        zipCode: String
    },
    createdAt: {
        type: Date,
        default: Date.now
    }
});

// Hash password before saving
userSchema.pre('save', async function(next) {
    if (!this.isModified('password')) {
        return next();
    }
    this.password = await bcrypt.hash(this.password, 10);
    next();
});

```

```
});  
  
// Compare password method  
userSchema.methods.comparePassword = async function(candidatePassword)  
{  
  return await bcrypt.compare(candidatePassword, this.password);  
};  
  
module.exports = mongoose.model('User', userSchema);
```

CHAPTER 8 : SCREENSHOT

 Car Wash

Services Login Register

Welcome to Car Wash Booking System

Book your car wash service online - Quick, Easy, and Convenient

Browse ServicesGet Started

Welcome to Car Wash Booking System

Book your car wash service online - Quick, Easy, and Convenient

 Car Wash

Services Login Register

Why Choose Us?



Save Time

Book your appointment online and avoid waiting in queues



Find Nearby

Search and book services from car wash centers near you



Easy Payment

Multiple payment options for your convenience



Quality Service

Read reviews and ratings from other customers

Register

Full Name

Email

Phone Number

I want to

Confirm Password

Street Address

City

State

Zip Code

Register

Already have an account? Login here

 Car Wash

Services Login Register

Login

Email

*****@gmail.com

Password

Login

Don't have an account? Register here

My Bookings

View and manage your car wash appointments

[All](#)
[Pending](#)
[Confirmed](#)
[Completed](#)
[Cancelled](#)

car wash

completed

Date	Time	Amount	Payment Status	Vehicle
December 18, 2025	09:00 - 09:30	\$100	pending	toyota innova 2019
Location abcd				

[View Details](#)

Our Services

Find the perfect car wash service for your vehicle

Filter Services

City

Category

All Categories

Min Price

Max Price

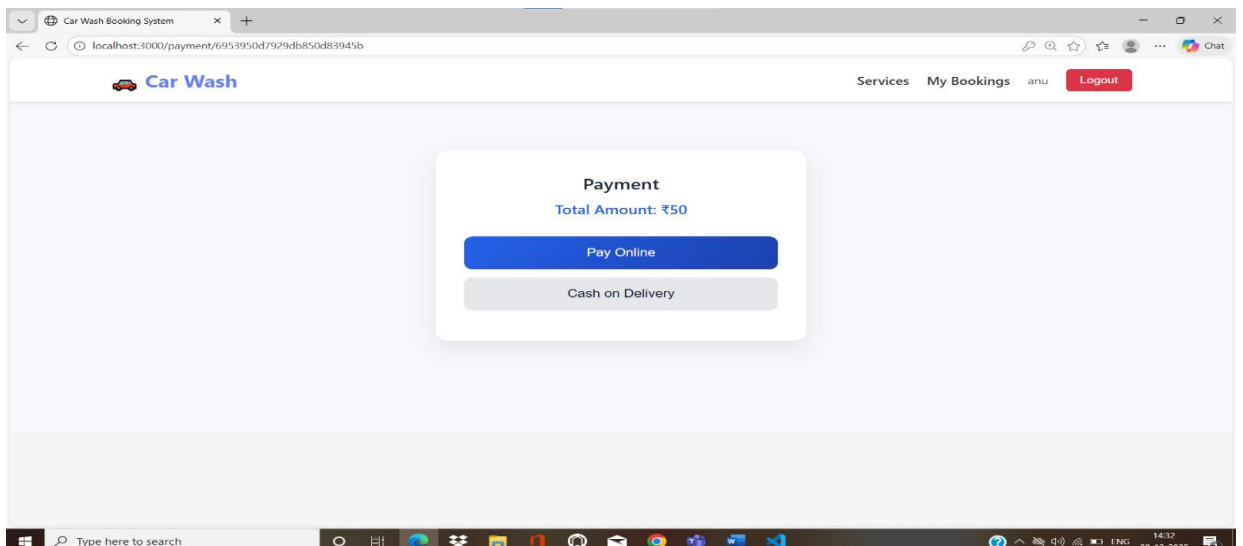
premium

car wash

[automatic car washing](#)
\$100

⌚ 30 min

📍 N/A



CHAPTER 9 : CONCLUSION

CleanRidex project successfully delivers a modern, efficient, and user-friendly Car Wash Booking System that overcomes the limitations of traditional manual booking methods. By digitizing the entire booking and service management process, the system provides a convenient platform for customers, service providers, and administrators to interact seamlessly.

Throughout the project, all phases of the Software Development Life Cycle (SDLC) were carefully followed, including requirement analysis, system design, implementation, testing, and maintenance. The system was designed using standard software engineering principles and implemented using the MERN stack, ensuring scalability, flexibility, and high performance.

CleanRidex enables users to register, browse services, book appointments, and track booking status with ease. Service providers can manage services and bookings efficiently, while administrators can monitor system activities and maintain overall control. Proper testing ensured system reliability, security, and accuracy, and maintenance strategies were defined to support long-term system stability.

In conclusion, CleanRidex meets its objectives by improving customer convenience, reducing waiting time, enhancing operational efficiency, and promoting digital transformation in the car wash industry. The project demonstrates the practical application of web technologies and software engineering concepts and serves as a strong foundation for future enhancements such as online payments, real-time notifications, mobile app support, and advanced analytics

CHAPTER 10 : REFERENCES

- Ian Sommerville, Software Engineering, 10th Edition, Pearson Education, 2016.
- Roger S. Pressman and Bruce R. Maxim, Software Engineering: A Practitioner's Approach, 8th Edition, McGraw-Hill Education, 2014.
- MongoDB Inc., MongoDB Documentation,
- Available at: <https://www.mongodb.com/docs>
- Meta Platforms Inc., React Official Documentation,
- Available at: <https://react.dev>
- OpenJS Foundation, Node.js Documentation,
- Available at: <https://nodejs.org/en/docs>
- Express.js Team, Express.js Documentation,
- Available at: <https://expressjs.com>
- Object Management Group (OMG), Unified Modeling Language (UML) Specification,
- Available at: <https://www.omg.org/spec/UML>
- IEEE Computer Society, IEEE Standard for Software and System Test Documentation, IEEE Std 829-2008.
- TutorialsPoint, Software Engineering Tutorials,
- Available at: https://www.tutorialspoint.com/software_engineering